LINEAR ALGEBRA FOR MACHINE LEARNING – LAB

Practical Manual

Course Code: BSAILB302P
Course Title: Linear Algebra for Machine Learning – Lab
Course Credit: 1
Total Contact Hours: 30 Hrs

PRACTICAL 1

Aim: Perform basic vector operations such as vector addition, scalar multiplication, and dot product using NumPy.

Description:

Vectors are one of the most basic mathematical structures used in machine learning and data science. A vector can be considered as an ordered collection of numbers. In Python, NumPy arrays are commonly used to represent vectors because they support fast and convenient mathematical operations.

In this practical, two vectors are created using NumPy. Vector addition is performed by adding the corresponding elements of the two vectors. Scalar multiplication means multiplying every element of a vector by a fixed number. The dot product is obtained by multiplying corresponding elements and adding the results.

These operations are important in many machine learning algorithms. For example, dot products are used in linear regression, neural networks, similarity calculations, and many optimization methods. Understanding vector operations provides a foundation for studying more advanced topics such as matrices, eigenvectors, PCA, and machine learning models.

Python Code:

```
import numpy as np

A = np.array([2, 4, 6])
B = np.array([1, 3, 5])

addition = A + B

scalar = 3
scalar_multiplication = scalar * A

dot_product = np.dot(A, B)

print("Vector A:", A)
print("Vector B:", B)
print("Addition:", addition)
print("Scalar Multiplication:", scalar_multiplication)
print("Dot Product:", dot_product)
```

Output:

```
Vector A: [2 4 6]
Vector B: [1 3 5]
Addition: [ 3  7 11]
Scalar Multiplication: [ 6 12 18]
Dot Product: 44
```

PRACTICAL 2

Aim: Calculate vector norms and distances using NumPy.

Description:

A vector norm is a measure of the size or length of a vector. Norms are widely used in machine learning to measure magnitude, calculate distances, normalize data, and understand the size of model parameters. Two commonly used norms are the L1 norm and L2 norm.

The L1 norm is calculated as the sum of the absolute values of the vector elements. The L2 norm, also known as the Euclidean norm, represents the usual geometric length of a vector. The Euclidean distance between two vectors is calculated as the L2 norm of their difference.

In this practical, NumPy's linalg.norm() function is used to calculate the L1 norm, L2 norm, and distance between two vectors. These concepts are important in clustering, nearest-neighbour methods, regularization, and other machine learning applications.

Python Code:

```
import numpy as np

A = np.array([3, 4])
B = np.array([6, 8])

l1_norm = np.linalg.norm(A, ord=1)
l2_norm = np.linalg.norm(A)
distance = np.linalg.norm(A - B)

print("L1 Norm:", l1_norm)
print("L2 Norm:", l2_norm)
print("Euclidean Distance:", distance)
```

Output:

```
L1 Norm: 7.0
L2 Norm: 5.0
Euclidean Distance: 5.0
```

PRACTICAL 3

Aim: Create and manipulate matrices using NumPy: addition, multiplication, transpose, and inverse.

Description:

A matrix is a rectangular arrangement of numbers in rows and columns. Matrices are fundamental to linear algebra and are extensively used in machine learning for representing datasets, transformations, model parameters, and computations.

In this practical, two matrices are created using NumPy. Matrix addition is performed by adding corresponding elements. Matrix multiplication is performed using the matrix multiplication operator @. The transpose changes rows into columns, while the inverse of a square non-singular matrix is a matrix that produces the identity matrix when multiplied by the original matrix.

These operations are useful in solving mathematical problems and understanding algorithms such as linear regression, neural networks, transformations, and dimensionality reduction.

Python Code:

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

addition = A + B
multiplication = A @ B
transpose = A.T
inverse = np.linalg.inv(A)

print("Addition:")
print(addition)

print("Multiplication:")
print(multiplication)

print("Transpose:")
print(transpose)

print("Inverse:")
print(inverse)
```

Output:

```
Addition:
[[ 6  8]
 [10 12]]

Multiplication:
[[19 22]
 [43 50]]

Transpose:
[[1 3]
 [2 4]]

Inverse:
[[-2.   1. ]
 [ 1.5 -0.5]]
```

PRACTICAL 4

Aim: Calculate determinant and rank using NumPy.

Description:

The determinant is a scalar value associated with a square matrix. It provides useful information about the matrix, including whether a square matrix is invertible. If the determinant is zero, the matrix is singular and does not have an ordinary inverse.

The rank of a matrix represents the number of linearly independent rows or columns. Rank is important for understanding the amount of independent information contained in a matrix. In machine learning and data science, rank can be used to identify redundant features and understand the structure of datasets.

In this practical, NumPy functions are used to calculate the determinant and rank of matrices. The example includes a full-rank matrix and a matrix whose rows are linearly dependent.

Python Code:

```
import numpy as np

A = np.array([[4, 2],
              [3, 1]])

B = np.array([[1, 2, 3],
              [2, 4, 6],
              [3, 6, 9]])

det_A = np.linalg.det(A)
rank_A = np.linalg.matrix_rank(A)
rank_B = np.linalg.matrix_rank(B)

print("Determinant of A:", det_A)
print("Rank of A:", rank_A)
print("Rank of B:", rank_B)
```

Output:

```
Determinant of A: -2.0
Rank of A: 2
Rank of B: 1
```

PRACTICAL 5

Aim: Solve systems of linear equations using NumPy.

Description:

A system of linear equations consists of two or more equations involving unknown variables. Such systems can be represented compactly in matrix form as AX = B, where A is the coefficient matrix, X contains the unknown variables, and B contains the constant values.

NumPy provides the function linalg.solve() to solve a system of linear equations when the coefficient matrix is suitable for obtaining a unique solution. In this practical, two simultaneous equations are converted into matrix form and solved using NumPy.

Solving linear systems is an important part of many machine learning and data science calculations. It appears in regression, optimization, numerical methods, and many mathematical models. The obtained solution can also be substituted back into the original equations to verify the result.

Python Code:

```
import numpy as np

A = np.array([[2, 1],
              [1, 3]])

B = np.array([5, 6])

solution = np.linalg.solve(A, B)

print("x =", solution[0])
print("y =", solution[1])

print("Verification:")
print(2 * solution[0] + solution[1])
print(solution[0] + 3 * solution[1])
```

Output:

```
x = 1.8
y = 1.4

Verification:
5.0
6.0
```

PRACTICAL 6

Aim: Compute eigenvalues and eigenvectors using NumPy.

Description:

Eigenvalues and eigenvectors are important concepts in linear algebra. For a square matrix A, an eigenvector v and its corresponding eigenvalue lambda satisfy the relationship Av = lambda v. This means that applying the matrix to an eigenvector changes its magnitude or direction according to the eigenvalue while preserving the eigenvector relationship.

Eigenvalues and eigenvectors are used in several machine learning and data science techniques. They are particularly important in Principal Component Analysis, where directions of maximum variance are related to eigenvectors of a covariance matrix.

In this practical, NumPy's linalg.eig() function is used to calculate the eigenvalues and eigenvectors of a matrix. The result can be used to understand important directions and scaling behaviour represented by the matrix.

Python Code:

```
import numpy as np

A = np.array([[4, 1],
              [2, 3]])

eigenvalues, eigenvectors = np.linalg.eig(A)

print("Eigenvalues:")
print(eigenvalues)

print("Eigenvectors:")
print(eigenvectors)

print("Verification:")
print(A @ eigenvectors)
```

Output:

```
Eigenvalues:
[5. 2.]

Eigenvectors:
[[ 0.7071 -0.4472]
 [ 0.7071  0.8944]]

Verification:
The matrix product satisfies A v = lambda v for each eigenvector.
```

PRACTICAL 7

Aim: Perform Principal Component Analysis (PCA) on a dataset using Python.

Description:

Principal Component Analysis (PCA) is a dimensionality reduction technique used to represent a dataset using a smaller number of meaningful variables called principal components. PCA transforms the original features into new directions that capture as much variation in the data as possible.

In this practical, a small dataset is standardized before PCA is applied. Standardization is useful when features have different scales. The PCA algorithm then identifies principal components and transforms the data into a lower-dimensional representation.

The explained variance ratio indicates how much of the total variation in the original dataset is represented by each principal component. PCA is widely used for visualization, feature reduction, noise reduction, and preprocessing before machine learning.

Python Code:

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

data = np.array([
    [2, 4, 1, 3],
    [3, 5, 2, 4],
    [4, 6, 3, 5],
    [5, 7, 4, 6],
    [6, 8, 5, 7],
    [7, 9, 6, 8]
])

scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)

pca = PCA(n_components=2)
data_pca = pca.fit_transform(data_scaled)

print("Principal Components:")
print(data_pca)

print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)

plt.scatter(data_pca[:, 0], data_pca[:, 1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Projection")
plt.show()
```

Output:

```
Principal Components:
[...]

Explained Variance Ratio:
[...]

A scatter plot of the observations in the two-dimensional PCA space is displayed.
```

PRACTICAL 8

Aim: Apply Singular Value Decomposition (SVD) for data compression.

Description:

Singular Value Decomposition (SVD) is a matrix factorization technique that represents a matrix as the product of three matrices: A = U Sigma V^T. The singular values in Sigma indicate the relative importance of different components of the data.

SVD can be used for dimensionality reduction and data compression. Instead of storing all components of a matrix, only the most important singular values and corresponding vectors can be retained. The original matrix can then be approximately reconstructed from these selected components.

In this practical, a small matrix is decomposed using NumPy. Only the first two singular values are retained to create a low-rank approximation. The original and compressed matrices are visualized using Matplotlib. SVD is useful in image compression, recommendation systems, noise reduction, and machine learning.

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt

A = np.array([
    [10, 10, 10, 10, 10],
    [10, 50, 50, 50, 10],
    [10, 50, 90, 50, 10],
    [10, 50, 50, 50, 10],
    [10, 10, 10, 10, 10]
])

U, S, VT = np.linalg.svd(A)

k = 2

compressed = (
    U[:, :k]
    @ np.diag(S[:k])
    @ VT[:k, :]
)

print("Singular Values:")
print(S)

print("Compressed Matrix:")
print(compressed)

plt.figure(figsize=(8, 3))

plt.subplot(1, 2, 1)
plt.imshow(A, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(compressed, cmap="gray")
plt.title("Compressed")
plt.axis("off")

plt.show()
```

Output:

```
Singular Values:
```

```
[...]

Compressed Matrix:
[...]

The original and compressed matrices are displayed as images.
```

PRACTICAL 9

Aim: Visualize vectors, matrices, and transformations using Matplotlib.

Description:

Visualization provides an intuitive way to understand linear algebra concepts. Vectors can be represented as arrows, while matrices can be interpreted as transformations that change the position or direction of vectors.

In this practical, a two-dimensional vector is created and a transformation matrix is applied to it. The resulting vector is calculated using matrix multiplication. Matplotlib's quiver() function is then used to display the original and transformed vectors.

Visualizing transformations helps students understand concepts such as scaling, rotation, reflection, and other geometric operations. Such ideas are useful in computer graphics, computer vision, robotics, and machine learning.

Python Code:

```
import numpy as np
import matplotlib.pyplot as plt

v = np.array([2, 1])

A = np.array([
    [2, 0],
    [0, 3]
])

transformed_v = A @ v

print("Original Vector:", v)
print("Transformation Matrix:")
print(A)
print("Transformed Vector:", transformed_v)

plt.quiver(0, 0, v[0], v[1], angles="xy",
           scale_units="xy", scale=1, label="Original")

plt.quiver(0, 0, transformed_v[0], transformed_v[1],
           angles="xy", scale_units="xy", scale=1,
           label="Transformed")

plt.xlim(0, 5)
plt.ylim(0, 5)
plt.grid()
plt.legend()
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Vector Transformation")
plt.show()
```

Output:

```
Original Vector: [2 1]

Transformation Matrix:
[[2 0]
 [0 3]]

Transformed Vector: [4 3]

A Matplotlib graph showing the original and transformed vectors is displayed.
```